



UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SICK

Master's Program: Master Intelligence Artificielle (DSBD & IA)

Module: Optimization Algorithms

**Optimization of Reinforcement
Learning Agents:
Impact of Optimization Algorithms
on DQN Training**

Prepared by:

Hafsa Elhilali

Jihed El Betti

Amina Bouazza

Supervised by:

Pr. Faouzia Benabbou

Academic Year: 2025 – 2026

Abstract

Reinforcement Learning (RL) presents unique optimization challenges due to the non-stationary and noisy nature of the data generated through agent-environment interaction. This project investigates the impact of five optimization algorithms—Stochastic Gradient Descent (SGD), SGD with Momentum, RMSprop, Adam, and AdamW—on the training of a Deep Q-Network (DQN) agent for the CartPole-v1 control task. A fixed DQN architecture, replay buffer, target network, and epsilon-greedy exploration schedule were used across all experiments, with only the optimizer varied between runs. Each optimizer was evaluated over 500 training episodes and further assessed across three random seeds (42, 123, 999) to examine sensitivity to random initialization and exploration noise. Performance was measured through episode reward, training loss, convergence speed, and training time. Results indicate that no single optimizer dominates across all criteria: SGD achieved the highest maximum reward but exhibited the greatest variance across seeds, SGD with Momentum obtained the best average final reward with comparatively moderate stability, while Adam produced the most consistent results across seeds despite a lower average reward. These findings highlight a trade-off between raw performance and training stability when selecting an optimizer for DQN agents, and underline the importance of multi-seed evaluation in reinforcement learning experiments. The limited number of seeds and episodes constrains the generalizability of these conclusions.

Contents

1	Introduction	1
2	Literature Review	2
3	Methodology	3
4	Experimental Setup	5
5	Results	6
6	Discussion	10
6.1	Interpretation of Results	10
6.2	Strengths and Weaknesses by Optimizer	11
7	Conclusion	12
	Appendix	14

List of Figures

1	Reward vs. episode: comparison of the five optimizers over 500 training episodes.	7
2	Training loss vs. episode: comparison of the five optimizers over 500 training episodes.	7
3	Average Training Time Across Three Random Seeds.	8
4	Mean Episode Reward Across Three Random Seeds.	8

List of Tables

1	Fixed configuration used across all experiments.	5
2	Software and hardware environment.	6
3	Multi-seed evaluation results across the three random seeds (42, 123, 999) for each optimizer.	9
4	Peak GPU memory usage per optimizer (single representative run, seed 42).	9
5	Complete hyperparameter configuration used across all experiments.	15

1 Introduction

Reinforcement Learning (RL) is a machine learning paradigm in which an autonomous agent learns to make sequential decisions through direct interaction with an environment. Unlike supervised learning, where a model is trained on a fixed dataset of labeled examples, an RL agent generates its own training data through trial and error, receiving a scalar reward signal after each action and adjusting its behavior to maximize cumulative reward over time. This interactive learning process has enabled RL to achieve notable results in domains such as robotic control, game playing, and autonomous decision-making, where explicit supervision is impractical or unavailable.

At the core of most modern RL algorithms lies a neural network trained through gradient-based optimization. The choice of optimization algorithm directly influences how efficiently and reliably this network converges toward an effective policy. While optimization has been extensively studied in supervised deep learning, its role in RL remains comparatively underexplored, despite the underlying learning dynamics differing substantially between the two settings. Optimizers such as SGD and its adaptive variants were originally developed and validated on stationary datasets, where the input distribution and target labels remain fixed throughout training; applying them to RL introduces additional challenges absent from the supervised setting.

A defining characteristic of RL training is the instability introduced by its non-stationary nature. As the agent’s policy evolves, the distribution of states and actions it encounters changes accordingly, so the training data is continuously shifting rather than fixed in advance. In value-based methods such as DQN, this instability is compounded by the fact that the target values used to compute the training loss are themselves derived from the network being trained, creating a moving-target problem. Small variations in gradient updates can therefore propagate into significant fluctuations in the learned policy, leading to oscillating training curves that are rarely observed to the same extent in supervised learning.

Given this instability, optimizer selection becomes a critical design decision rather than a secondary implementation detail. Different optimization algorithms handle gradient noise, learning rate adaptation, and parameter updates in fundamentally different ways, which can lead to substantial differences in learning speed, convergence reliability, and sensitivity to random initialization. Understanding these differences is essential for practitioners seeking to train RL agents efficiently and reproducibly, particularly in academic and resource-constrained settings where extensive hyperparameter tuning is not always feasible.

The objective of this project is to study the impact of optimization algorithms on the training of a DQN agent applied to the CartPole-v1 control task. Five widely used optimizers are considered: SGD, SGD with Momentum, RMSprop, Adam, and AdamW.

All experiments are conducted under identical conditions—network architecture, replay buffer configuration, exploration schedule, and training duration—so that the optimizer remains the only variable across comparisons. Beyond a single training run per optimizer, this work incorporates a multi-seed evaluation protocol to assess sensitivity to random initialization and exploration noise, a dimension often overlooked in smaller-scale RL studies but with direct implications for the reliability of reported results.

The contribution of this work is twofold. First, it provides a controlled empirical comparison of five optimization algorithms under a common DQN training pipeline, evaluated according to learning speed, reward stability, convergence behavior, and computational cost. Second, it examines the extent to which the observed performance of each optimizer depends on the choice of random seed, offering practical insight into the reproducibility of optimizer-based conclusions in RL experiments. The remainder of this report details the theoretical background, the experimental methodology, the results obtained, and their implications for optimizer selection in DQN training.

2 Literature Review

RL has been formalized as a framework in which an agent learns an optimal behavior policy through repeated interaction with an environment, guided by a scalar reward signal rather than explicit supervision [1]. Its theoretical foundations rest on the Markov Decision Process, which models the environment through states, actions, transition probabilities, and rewards. Classical tabular methods, such as Q-learning, estimate the value of state-action pairs directly, but do not scale to environments with large or continuous state spaces, since maintaining an explicit value table becomes computationally intractable [2]. This limitation motivated the integration of function approximation, and neural networks in particular, into RL algorithms.

The Deep Q-Network (DQN) algorithm addressed this scalability problem by using a deep neural network to approximate the action-value function, enabling RL agents to operate directly on high-dimensional observations [3]. Two mechanisms were introduced to stabilize training under this approximation scheme. The experience replay buffer stores past transitions and samples them randomly during training, breaking the temporal correlation between consecutive experiences. The target network, a delayed copy of the main network used to compute bootstrapped targets, mitigates the moving-target problem that arises when the same network is used both to predict and to generate its own training targets. Both components have since become standard practice in value-based deep RL [3].

Stochastic Gradient Descent remains the most fundamental optimization algorithm used to train neural networks, updating parameters in the direction opposite to the gradient of the loss function estimated on a mini-batch of data [4]. While computationally

simple and well understood theoretically, plain SGD converges slowly on ill-conditioned loss surfaces and is sensitive to the choice of learning rate, particularly when gradients vary substantially in magnitude across parameters. Momentum was introduced to accelerate convergence and dampen oscillations by accumulating an exponentially weighted moving average of past gradients [5], allowing the optimizer to maintain velocity in directions of consistent descent while reducing the impact of noisy or conflicting gradient components.

RMSprop was developed to address the limitation of a single global learning rate by adapting the effective step size for each parameter individually, based on a running average of squared gradients [6]. This adaptive scaling allows RMSprop to handle non-stationary objectives more gracefully than plain SGD, which made it a popular choice in early deep RL implementations, where gradient magnitudes vary considerably across training. Adam combines momentum with per-parameter adaptive learning rates by maintaining exponentially decaying averages of both the gradient and its squared value, used to compute bias-corrected update estimates [7]. This combination has made Adam one of the most widely adopted optimizers in deep learning, and a common default choice in deep RL, owing to its fast convergence and relative robustness to hyperparameter choices.

AdamW modifies the original Adam algorithm by decoupling the weight decay term from the gradient-based update, rather than incorporating it as an L2 penalty inside the gradient [8]. This decoupling was shown to improve generalization in supervised settings, and AdamW has since been adopted in various deep learning pipelines, including RL implementations, as a drop-in replacement for Adam.

Comparative studies in supervised deep learning generally find that adaptive optimizers such as Adam converge faster in early training compared to SGD, although SGD with a well-tuned learning rate and momentum can sometimes achieve comparable or superior final generalization, particularly in image classification [9]. In deep RL, however, comparative evidence remains more limited, and the interaction between optimizer choice and the noisy, non-stationary gradients characteristic of RL training is less thoroughly documented. This gap motivates the empirical comparison undertaken in this project.

3 Methodology

The CartPole-v1 environment, provided through the Gymnasium library, was selected as the experimental testbed due to its well-defined dynamics and widespread use as a benchmark for value-based RL algorithms. A pole is attached by an unactuated joint to a cart moving along a frictionless track, and the agent must apply forces to the cart to keep the pole balanced. An episode terminates when the pole deviates beyond a fixed angular threshold, the cart moves outside the track boundaries, or a maximum length of 500 time steps is reached—the latter representing the maximum achievable reward per episode.

The observation space consists of four continuous variables: cart position, cart velocity,

pole angle, and pole angular velocity, provided directly to the agent without preprocessing. The action space is discrete, with two possible actions corresponding to pushing the cart left or right. A reward of one is granted per time step during which the pole remains balanced, so the cumulative episode reward directly reflects the duration of successful control.

The DQN agent approximates the action-value function through a feedforward network composed of an input layer of four neurons, two fully connected hidden layers of 128 neurons each with ReLU activations, and an output layer of two neurons corresponding to the Q-values of the two actions. This architecture was deliberately kept simple, since the objective is to isolate the effect of the optimizer rather than to explore network capacity. The same architecture and weight initialization procedure were used across all experiments.

A replay buffer of fixed capacity stores transitions consisting of a state, an action, a reward, the resulting next state, and a termination flag. Mini-batches are sampled uniformly at random from this buffer during training rather than relying only on the most recent experience, which reduces temporal correlation between training samples and allows past experiences to be reused, improving sample efficiency and training stability. A target network, maintained as a separate copy of the policy network, is used exclusively to compute bootstrapped Bellman targets and is not updated through gradient descent; instead, its weights are periodically synchronized with the policy network, which prevents the target values from shifting at the same rate as the network being optimized.

Exploration follows an epsilon-greedy strategy: the agent selects a random action with probability epsilon and otherwise selects the action with the highest predicted Q-value. Epsilon is initialized at a high value to encourage exploration early in training and decays after each episode according to a fixed multiplicative schedule down to a small minimum value, held constant across all experiments so that differences in performance cannot be attributed to variations in exploration.

All hyperparameters governing the environment, network architecture, replay buffer, target network updates, and exploration schedule were fixed across every experiment, as summarized in Table 1. This is central to the methodology: since the goal is to isolate the specific contribution of the optimizer, every other component of the pipeline was held strictly identical between runs, with the optimizer being the only variable allowed to change.

Table 1: Fixed configuration used across all experiments.

Parameter	Value
Hidden layer size	128 neurons per layer
Number of hidden layers	2
Activation function	ReLU
Replay buffer capacity	10,000 transitions
Batch size	64
Discount factor (γ)	0.99
Epsilon (start / minimum / decay)	1.0 / 0.01 / 0.995 per episode
Target network update frequency	every 10 episodes
Learning rate (all optimizers)	1×10^{-3}
Number of training episodes	500
Random seeds evaluated	42, 123, 999

Five optimizers were compared under this fixed pipeline: SGD without momentum, SGD with a momentum coefficient of 0.9, RMSprop, Adam, and AdamW. All were configured with the same learning rate, and no per-optimizer tuning was performed, isolating the effect of the update rule itself. Each optimizer was applied exclusively to the policy network, while the target network continued to be updated solely through periodic weight copying.

The experimental protocol proceeded in two stages. First, each optimizer was trained for 500 episodes using a single fixed seed, providing an initial comparison of learning curves, training loss, and computational cost. Second, to assess whether observed differences could be attributed to random initialization and stochastic exploration rather than the optimizer itself, each optimizer was retrained using three seeds (42, 123, 999). For each optimizer-seed combination, the final episode reward, average reward, maximum reward, and total training time were recorded, and the mean and standard deviation of these metrics were computed for each optimizer.

4 Experimental Setup

All experiments were implemented in Python, using PyTorch for the DQN agent, replay buffer, and training loop, and Gymnasium for the CartPole-v1 environment. Training was carried out on the Kaggle platform using its GPU accelerator, with PyTorch executing tensor operations through CUDA whenever available and falling back to CPU otherwise. This acceleration was particularly relevant given that the combination of optimizer comparison and multi-seed evaluation required fifteen independent 500-episode training runs. Table 2 summarizes the software and hardware environment.

Table 2: Software and hardware environment.

Component	Description
Programming language	Python
Deep learning framework	PyTorch
RL environment library	Gymnasium (CartPole-v1)
Execution platform	Kaggle Notebooks
Hardware accelerator	GPU (CUDA-enabled), CPU fallback

The full training configuration, listed previously in Table 1, was held identical across all optimizers and all random seeds, so that any observed difference in performance could be attributed to the optimization algorithm rather than to variation in the experimental setup. A learning rate of 1×10^{-3} , a discount factor of 0.99, a batch size of 64, a replay buffer capacity of 10,000 transitions, and a target network update frequency of every 10 episodes were applied uniformly. The discount factor gives substantial weight to long-term reward, appropriate for a task where the objective is to sustain balance over an extended sequence of steps. The batch size and buffer capacity offer a reasonable trade-off between gradient estimate stability, sample diversity, and computational cost, while the target update frequency balances stability of the bootstrapped targets against the network’s ability to track policy improvement.

Each optimizer was evaluated across three independent training runs using seeds 42, 123, and 999, controlling network initialization as well as the stochastic elements of environment reset and action exploration. This produced a total of fifteen independent training runs, from which the mean and standard deviation of the recorded performance metrics were computed for each optimizer.

5 Results

This section reports the outcomes of the experiments comparing the five optimization algorithms, from both the single-seed training runs and the multi-seed evaluation across the three random seeds. The presentation is limited to a factual description of the figures and table; interpretation is provided in the Discussion.

Figure 1 shows the total reward per episode over 500 training episodes for each optimizer. All curves start at low reward values, consistent with the near-random behavior of the agent while epsilon is close to its maximum, and exhibit an overall upward trend accompanied by substantial episode-to-episode fluctuations, with several optimizers reaching peak rewards close to the maximum value of 500 at different points. The SGD and SGD with Momentum curves display the most pronounced peaks late in training, while RMSprop, Adam, and AdamW remain comparatively lower and more contained in

amplitude.

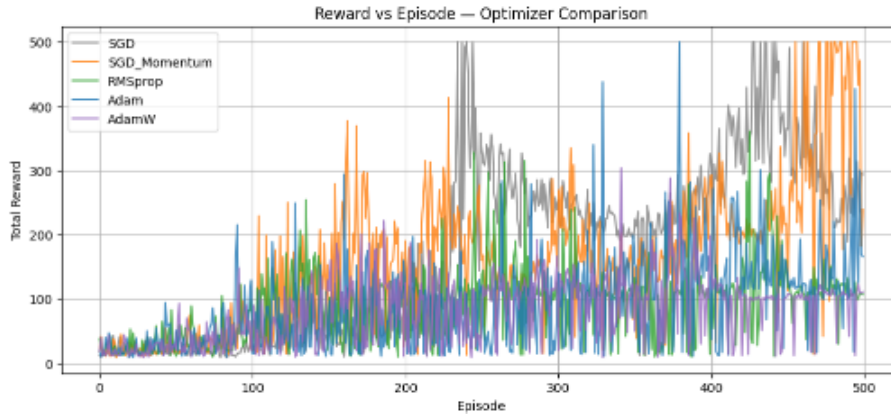


Figure 1: Reward vs. episode: comparison of the five optimizers over 500 training episodes.

Figure 2 presents the average training loss per episode. All five curves begin at relatively low values and generally increase in magnitude as training advances, rather than decreasing monotonically. SGD shows the most distinct pattern, rising sharply between roughly episode 100 and 250 to values above 3.5 before decreasing and later increasing again. The remaining optimizers display comparatively lower loss values throughout, with intermittent spikes for SGD with Momentum and RMSprop in later episodes.

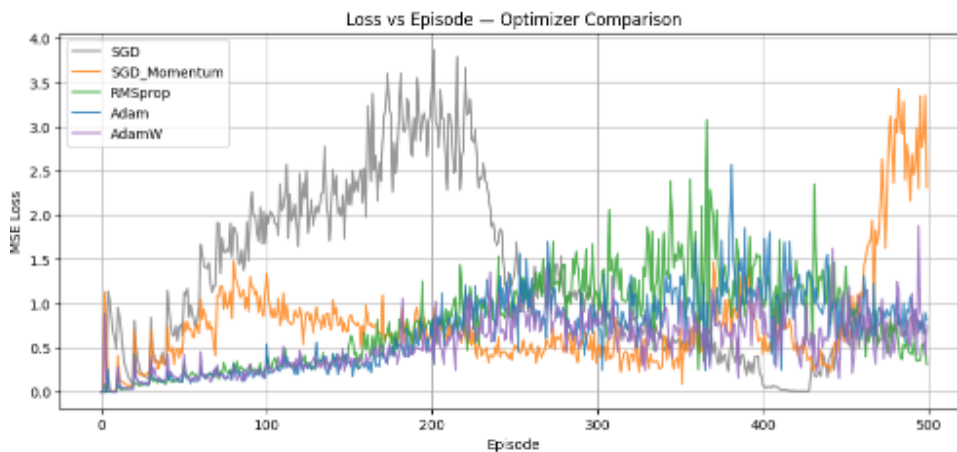


Figure 2: Training loss vs. episode: comparison of the five optimizers over 500 training episodes.

Figure 3 reports the mean training time, in seconds, required to complete a 500-episode run, averaged across the three seeds. SGD required the longest average training time, followed by SGD with Momentum, AdamW, RMSprop, and Adam, with Adam corresponding to the shortest mean training time.

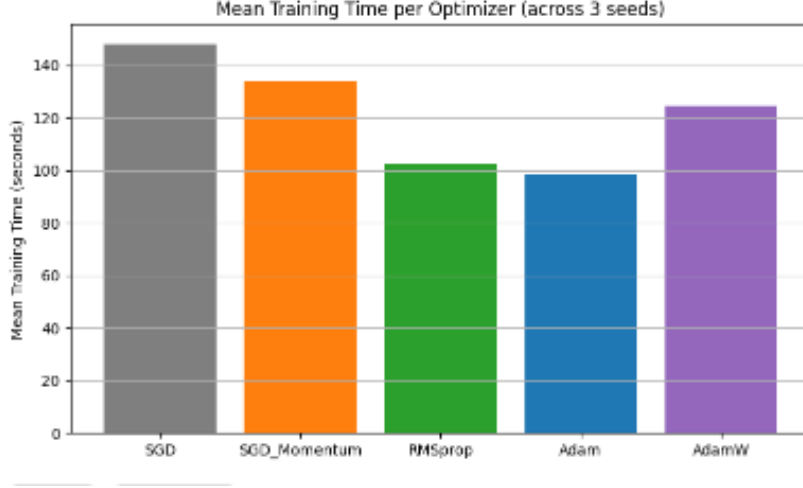


Figure 3: Average Training Time Across Three Random Seeds.

Figure 4 shows the mean reward achieved by each optimizer across the three seeds, with error bars representing the standard deviation. SGD obtained the highest mean reward but also the largest error bar, indicating the greatest variability. SGD with Momentum and AdamW show intermediate mean rewards, with AdamW displaying a wider error bar than SGD with Momentum. RMSprop and Adam show lower mean rewards, with Adam exhibiting the narrowest error bar of the five.

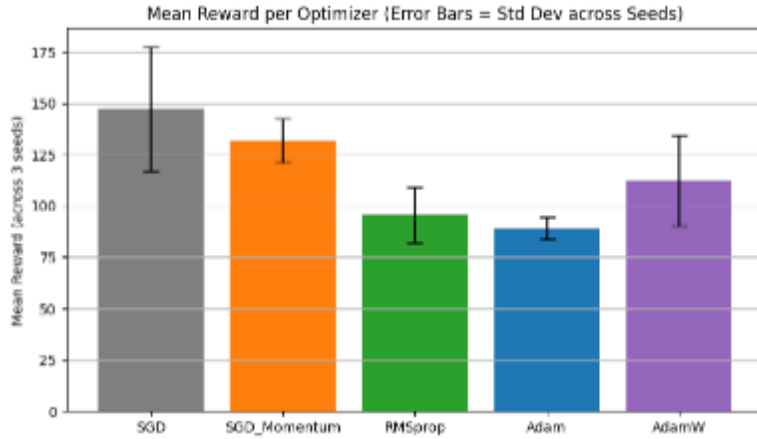


Figure 4: Mean Episode Reward Across Three Random Seeds.

Table 3 summarizes the multi-seed evaluation for each optimizer. SGD shows the highest mean reward (147.31) and the highest mean maximum reward (500.00), together with the highest standard deviation (30.30). SGD with Momentum shows the highest mean final reward (354.67), with a mean reward of 131.84 and a standard deviation of 10.66. RMSprop shows a mean reward of 95.50, a mean final reward of 332.33, and a standard deviation of 13.70. Adam shows the lowest mean reward (89.39) and lowest mean final reward (80.67), together with the lowest standard deviation (5.26) and shortest

mean training time (98.37 s). AdamW shows a mean reward of 112.31, a mean final reward of 156.67, and a standard deviation of 21.90.

Table 3: Multi-seed evaluation results across the three random seeds (42, 123, 999) for each optimizer.

Optimizer	Mean Reward	Std Reward	Mean Final Reward	Mean Max Reward
SGD	147.31	30.30	267.00	500.00
SGD_Momentum	131.84	10.66	354.67	458.33
RMSprop	95.50	13.70	332.33	395.33
Adam	89.39	5.26	80.67	485.00
AdamW	112.31	21.90	156.67	453.00

Table 4: Peak GPU memory usage per optimizer (single representative run, seed 42).

Optimizer	Peak GPU Memory (MB)
SGD	18.01
SGD_Momentum	18.08
RMSprop	18.08
Adam	18.15
AdamW	18.15

Table 4 reports the peak GPU memory usage recorded for each optimizer during a single representative training run. Memory was profiled on one run per optimizer rather than across all three seeds, since GPU memory usage showed negligible seed-to-seed variation in preliminary checks for a network of this size. The measured values range narrowly from 18.01 MB (SGD) to 18.15 MB (Adam and AdamW), a difference of less than 1%. This is consistent with the underlying mechanics of each optimizer: SGD maintains no additional state beyond the current gradient, while Momentum, RMSprop, Adam, and AdamW store one or two extra tensors per parameter (velocity and/or second-moment estimates). Given the small size of the network used in this study (approximately 9,000 parameters), this additional state adds only a few kilobytes of overhead, which is negligible compared to the fixed cost of storing the network itself, the replay buffer, and CUDA’s runtime allocation overhead. This finding indicates that, for small-scale DQN architectures such as the one used here, optimizer choice has no meaningful impact on memory requirements, and the decision should instead be guided by the performance and stability trade-offs discussed in Section 6.

6 Discussion

6.1 Interpretation of Results

The results reveal that no single optimizer dominates on every evaluation criterion, and that the ranking depends strongly on which aspect of training performance is considered—consistent with the fact that each algorithm was designed to address a specific limitation of gradient-based training rather than to provide a universally superior update rule.

In terms of learning speed, all five optimizers required a comparable number of episodes before showing a clear upward trend in reward, with meaningful improvement generally visible only after the first hundred episodes. This delay is largely attributable to the epsilon-greedy schedule rather than the optimizer, since the agent behaves close to randomly while epsilon remains high. Differences between optimizers become more apparent later in training, once the replay buffer has accumulated sufficient informative transitions.

Regarding convergence, SGD achieved the highest mean maximum reward and the highest overall mean reward, indicating that it reached strong policies at certain points during training; however, its mean final reward (267.00) remained considerably lower than its maximum, suggesting the learned policy did not persist at its best level by the end of training. SGD with Momentum achieved the highest mean final reward (354.67), suggesting a better ability to sustain a good policy toward the end of training, and RMSprop also achieved a relatively high mean final reward (332.33). Adam showed the lowest mean final reward (80.67) despite a high mean maximum reward (485.00), suggesting its performance peaked earlier and did not persist under the fixed learning rate used here.

Stability, assessed through the standard deviation of reward across seeds, presents a different picture. Adam obtained the lowest standard deviation (5.26), the most consistent and least seed-dependent result, while SGD obtained the highest (30.30)—more than five times larger. SGD with Momentum showed a comparatively low standard deviation (10.66), positioning it between Adam and SGD in reliability, while AdamW showed a higher standard deviation (21.90) than either. This pattern suggests that adaptive per-parameter learning rates contribute to more predictable outcomes, whereas the fixed global learning rate of plain SGD makes it more sensitive to the specific gradient trajectory of a given run.

This variance across seeds carries an important methodological implication: a comparison based on a single run per optimizer would have led to different, and potentially misleading, conclusions. Under the single-seed results, SGD and SGD with Momentum appeared to substantially outperform the remaining optimizers, largely due to occasional high-reward episodes late in training. Averaged over three seeds, this apparent advantage is tempered by the high variability of SGD, while Adam and AdamW show more modest but more reproducible outcomes—reinforcing the argument that optimizer comparisons

in RL should not rely on isolated runs, given the additional stochasticity introduced by exploration and environment resets.

Regarding computational cost, Adam required the shortest mean training time (98.37 s), closely followed by RMSprop (102.41 s), while SGD required the longest (148.12 s). Since the network architecture and number of gradient updates per episode were identical across optimizers, this difference is primarily attributable to variation in episode length—longer episodes generate more environment steps and replay buffer insertions—rather than to the computational cost of the update rule itself.

These differences relate directly to the underlying mechanics of each optimizer. Plain SGD updates all parameters using a single, fixed learning rate with no memory of past gradients, making it sensitive to the scale and direction of the current gradient estimate; in the non-stationary setting of DQN training, this can produce occasionally strong but inconsistent updates, consistent with the high variance observed here. Momentum partially mitigates this by smoothing the update direction over time, likely contributing to the improved final-reward stability of SGD with Momentum. RMSprop and Adam introduce per-parameter adaptive learning rates based on recent gradient magnitudes, which should in principle improve robustness to changing gradient scales; the low variance of Adam is consistent with this, although its comparatively low final reward suggests the fixed learning rate used here may not have been optimal for its adaptive mechanism. AdamW behaved similarly to Adam in computational cost but showed higher variance, suggesting a more limited benefit from decoupled weight decay in the absence of explicit regularization in this setup.

6.2 Strengths and Weaknesses by Optimizer

SGD. Its main strength is simplicity: no additional state is maintained beyond the current gradient, resulting in minimal computational and memory overhead. In this study, SGD achieved the highest mean and maximum reward, showing it is capable of discovering strong policies. Its principal weakness is a lack of gradient memory or adaptive scaling, which produced the highest variance across seeds (30.30) and a mean final reward well below its maximum (267.00 vs. 500.00), indicating an inability to reliably retain a good policy once found.

SGD with Momentum. Adding momentum partially smooths the update direction, which translated into the highest mean final reward observed (354.67) and a moderate standard deviation (10.66) — a better balance between performance and stability than plain SGD. Its remaining weakness is that it still relies on a single global learning rate, leaving it more sensitive to gradient scale than fully adaptive methods.

RMSprop. Its per-parameter adaptive scaling is well suited to the non-stationary targets generated by DQN, and it achieved a reasonably high final reward (332.33) with

moderate variance (13.70). Its main limitation is the absence of a momentum term, which can leave it less able to smooth out noisy updates compared to Adam.

Adam. Combining momentum with adaptive learning rates, Adam produced by far the most consistent results across seeds (standard deviation of 5.26, more than five times lower than SGD) and required the shortest training time (98.37 s). Its main weakness in this study is a comparatively low final reward (80.67), suggesting that under the fixed learning rate used here, Adam converged early but did not continue improving toward the end of training.

AdamW. By decoupling weight decay from the gradient update, AdamW is designed to improve generalization; however, in the absence of explicit weight decay in this experimental setup, its behavior closely resembled Adam’s while exhibiting higher variance (21.90). This suggests that, under this configuration, AdamW’s theoretical advantage did not translate into a measurable benefit for this task.

Several limitations should be acknowledged. Only three random seeds were used per optimizer, a relatively small sample from a statistical standpoint. All optimizers were evaluated using the same fixed learning rate, without optimizer-specific tuning, so some optimizers—particularly plain SGD—might perform differently under a better-suited learning rate. The experiments were limited to a single, relatively simple environment and a single, shallow network architecture, and training was limited to 500 episodes, which may not have been sufficient for all optimizers to reach a stable converged policy. These limitations suggest that the conclusions of this study should be interpreted as specific to the experimental conditions considered, rather than as general claims about optimizer performance in reinforcement learning as a whole.

7 Conclusion

This project examined the impact of optimization algorithms on the training of a DQN agent applied to CartPole-v1, comparing SGD, SGD with Momentum, RMSprop, Adam, and AdamW. Since RL training data is generated by an evolving policy rather than drawn from a fixed distribution, the choice of optimizer was expected to influence not only learning speed but also the stability and reproducibility of the resulting policy. The study isolated this effect by holding the network architecture, replay buffer, exploration schedule, and all other hyperparameters constant, allowing the optimizer to be the sole variable under comparison.

The experimental work proceeded in two stages: an initial single-seed comparison, followed by a multi-seed evaluation across three seeds (42, 123, 999) that enabled the computation of mean and standard deviation for each performance metric. The results show that no optimizer performed best on every criterion. SGD achieved the highest mean and maximum reward but also the greatest variability, indicating limited reproducibility.

SGD with Momentum obtained the highest mean final reward, suggesting a better ability to sustain a good policy by the end of training. Adam produced the most consistent results across seeds, with the lowest standard deviation and shortest training time, although its average and final rewards were lower under the fixed learning rate used in this study. These findings point to a trade-off between raw performance and training stability that should be considered when selecting an optimizer for DQN-based agents.

Based on these observations, optimizer choice for DQN training should be guided by the relative importance of peak performance versus reproducibility. Where consistent, predictable training is prioritized—particularly when resources for repeated experimentation are limited—Adam or AdamW appear preferable due to their lower sensitivity to seed variation. Where achieving a high-performing policy is the primary concern and multiple runs can be conducted to mitigate variance, SGD with Momentum offers a reasonable balance between final performance and stability. These recommendations are specific to the experimental conditions examined here and should not be generalized without further validation.

Future work could extend this study by evaluating a larger number of random seeds for a more statistically robust estimate of variance; testing optimizer-specific learning rates rather than a single fixed value; extending the comparison to more complex environments and alternative RL algorithms such as Double DQN or Actor-Critic methods; and incorporating longer training durations to clarify whether the instability observed late in training for certain optimizers is transient or persistent.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.
- [3] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [4] H. Robbins and S. Monro, “A stochastic approximation method,” *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.
- [5] B. T. Polyak, “Some methods of speeding up the convergence of iteration methods,” *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [6] T. Tieleman and G. Hinton, “Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude,” *COURSERA: Neural Networks for Machine Learning*, 2012.
- [7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [8] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, 2019.
- [9] A. C. Wilson, R. Roelofs, M. Stern, N. Srebro, and B. Recht, “The marginal value of adaptive gradient methods in machine learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

Appendix

A. Code Repository

The complete implementation of the DQN agent, the replay buffer, the training loop, and the optimizer comparison experiments is publicly available at the following repository:

[<https://github.com/Hafsa58/rl-optimizer-comparison>]

The repository includes the source code, instructions for reproducing the experiments, and the raw results obtained across the five optimizers and three random seeds.

B. Kaggle Notebook

An executable Jupyter notebook demonstrating the full training pipeline, including environment setup, model definition, training loop, and result visualization, is available on Kaggle at the following link:

[<https://www.kaggle.com/code/hafsa991/optimisation-f>]

This notebook can be run directly on the Kaggle platform with GPU acceleration enabled, and reproduces all figures and tables presented in this report.

C. Complete Hyperparameter Table

Table 5 reproduces the complete set of hyperparameters used across all experiments, consolidating the values presented in Table 1.

Table 5: Complete hyperparameter configuration used across all experiments.

Parameter	Value
Hidden layer size	128 neurons per layer
Number of hidden layers	2
Activation function	ReLU
Replay buffer capacity	10,000 transitions
Batch size	64
Discount factor (γ)	0.99
Epsilon (start / minimum / decay)	1.0 / 0.01 / 0.995 per episode
Target network update frequency	every 10 episodes
Learning rate (all optimizers)	1×10^{-3}
Number of training episodes	500
Random seeds evaluated	42, 123, 999